School of Technology,Design and Computer Application
College Of Technology
Bachelor of Technology (Computer Engineering)

Semester:	6	**Academic Year:**	2025-26
Course Name:	Artificial Intelligence and Machine Learning	**Course Code:**	1010043341

Question bank

Sr. No.	Question Text	Marks	CO
Unit 1 : Introduction to AI			
1	Define: AI. List down the application of AI and explain any one.	8	CO1
2	List down the problem characteristics of AI.	5	CO1
3	Analyze (a) 8-puzzle, (b) Chess and (c) Tower of Hanoi problems with respect to the following problem characteristics: I. Is the problem decomposable? II. Can solution steps be ignored? III. Is the good solution absolute or relative? IV. Is the solution state or a path? V. What is the role of knowledge?	5	CO1
4	Difference between: ANN and BNN.	5	CO1
5	Explain different types of learning with examples.	4	CO1
6	Difference between: Supervised and Unsupervised Learning.	4	CO1
7	Explain the elements of data science.	5	CO1

8	Explain Data Visualization techniques.	5	CO1
Unit 2 :Exploratory Data Analysis			
9	What is exploratory data analysis?	5	CO2
10	Explain descriptive statistics.	4	CO2
11	Difference between data and histogram	5	CO2
12	Explain 3Ms.	3	CO2
13	Explain Measure of Dispersion	5	CO2
14	Explain 5 number summary(Box-Plot Summary)	4	CO2
Unit 3 : Inferential Statistics			
15	Explain the concept of probability and its type.	5	CO2
16	Difference between: Descriptive and Inferential Statistics	3	CO2
17	Explain types of Inferential Statistics.	5	CO2
18	Explain Random Variables with its type.	3	CO2
19	Explain the Central Limit theorem with its rules.	5	CO2
20	Explain sampling distribution with its types	4	CO2
21	Explain Cross Validation with its types	8	CO2
22	Explain Bayesian theorem with its importance	5	CO2
Unit 4 : Supervised Learning			
23	What is Supervised learning? Explain the problems in Supervised learning.	5	CO3
24	Difference between: Classification and Regression	4	CO3
25	Explain Linear Regression.	3	CO3
26	Explain Logistic Regression.	5	CO3
27	Explain Polynomial Regression.	3	CO3
28	Explain Decision Tree	5	CO3
29	Explain Random Forest	5	CO3

30	Explain Naive Bayes	3	CO3
31	Explain SVM.	4	CO3
Unit 5 : Unsupervised Learning			
32	Explain Unsupervised Learning? Explain the problems in Unsupervised learning	3	CO3
33	Explain K-means.	4	CO3
34	Explain PCA.	3	CO3
35	Explain Different Libraries of Python	5	CO3
Unit 6 : Neural Nets			
36	Explain neural networks. List down the applications, advantages,and limitations of NN.	5	CO4
37	Explain Topologies of NN.	8	CO4
38	Explain perceptron.	4	CO4
39	Explain feed forward NN.	8	CO4
40	Explain multi-layer perceptron.	5	CO4
41	Explain CNN.	8	CO4
42	Explain RNN	8	CO4
43	Define: Training, Testing and validation	8	CO4
44	Define Parameter Estimation: MLE,MAP, Bayesian estimation.	5	CO4
Unit 7 : Deep Learning (Artificial Neural Network)			
45	What is deep learning? List down the applications of deep learning and explain any one.	8	CO5
46	Write down the Steps to build ANN.	8	CO5
47	Define optimizers. List down the types of optimizers.	8	CO5
48	Explain Gradient Descent	8	CO5
49	Explain Stochastic Gradient Descent	4	CO5

Course Coordinator **Head of Department**

SILVER OAK UNIVERSITY

School of Technology, Design and Computer Application

B.Tech Computer Engineering — 6th Semester

Artificial Intelligence & Machine Learning

Course Code: 1010043341

★ COMPLETE QUESTION BANK WITH SOLUTIONS ★

All 7 Units | 49 Questions | Academic Year 2025–26

▶ UNIT 1: INTRODUCTION TO ARTIFICIAL INTELLIGENCE

Q1. Define AI. List down the applications of AI and explain any one. [8 Marks]

Definition of Artificial Intelligence

Artificial Intelligence (AI) is the branch of computer science that focuses on building systems capable of performing tasks that typically require human intelligence — such as learning from experience, understanding natural language, reasoning, solving problems, recognising patterns, and making decisions. The term was coined by John McCarthy at the Dartmouth Conference in 1956.

```
  AI = Techniques that enable machines to mimic cognitive functions of the
human mind
```

AI systems are powered by algorithms, data, and computational hardware. AI is broadly categorised as:

- **Narrow AI (Weak AI):** Designed for a specific task (e.g., chess engine, face recognition, Siri).
- **General AI (Strong AI):** Hypothetical AI that can perform ANY intellectual task a human can do.
- **Super AI:** Hypothetical AI that surpasses human intelligence in all aspects.

AI Hierarchy Diagram

```
                  +---------------------+
                  |   ARTIFICIAL        |
                  |   INTELLIGENCE      |
                  +----------+----------+
            +-----+          |          +-----+
            |   Machine      |   Expert       |
            |   Learning     |   Systems      |
            +----+------+    |    +------+----+
                 |      |    |    |      |
            Deep  NLP  CV  Robotics Search Planning
           Learn
```

Applications of AI

Domain	Application	Example Systems
Healthcare	Diagnosis, drug discovery, robotic surgery	DeepMind Eye AI, IBM Watson
Finance	Fraud detection, algorithmic trading, risk assessment	Stripe Radar, Bloomberg AIM
Autonomous Vehicles	Self-driving cars, drones, path planning	Tesla FSD, Waymo
Natural Language	Chatbots, translation, sentiment analysis	GPT-4, Google Translate
Education	Personalised learning, automated grading	Duolingo, Coursera AI
Entertainment	Recommendation engines, content generation	Netflix, Spotify, DALL-E
Manufacturing	Predictive maintenance, quality control	Siemens MindSphere

Domain	Application	Example Systems
Cybersecurity	Threat detection, anomaly identification	Darktrace, CrowdStrike

Detailed Explanation: AI in Healthcare

AI in Healthcare is one of the most transformative applications. AI-powered diagnostic systems analyse medical images — X-rays, MRI, CT scans — with accuracy equal to or exceeding that of specialist doctors.

- **Medical Imaging:** Google DeepMind can detect 50+ eye diseases from retinal scans. Pathology AI detects cancer cells in tissue biopsies.
- **Drug Discovery:** AlphaFold (DeepMind) predicted 3D structures of 200 million proteins, accelerating drug target identification.
- **Robotic Surgery:** da Vinci Surgical System uses AI-guided robotic arms for minimally invasive surgery with sub-millimetre precision.
- **Predictive Analytics:** AI predicts patient deterioration, hospital readmissions, and sepsis onset hours before clinical symptoms appear.

Q2. List down the problem characteristics of AI. [5 Marks]

Problem Characteristics of AI

When solving a problem using AI, the following characteristics define the nature of the problem and determine the most appropriate solution technique:

Characteristic	Description	Example
1. Decomposability	Can the problem be broken into independent sub-problems?	Chess: analyse individual endgames
2. Ignorability of Steps	Can solution steps be undone/skipped without consequences?	Theorem proving: yes. Surgery: no
3. Absolute vs Relative Solution	Is there one correct answer (absolute) or just a better one (relative)?	Sorting = absolute. Chess strategy = relative
4. State vs Path Solution	Is the final state important, or the sequence of steps?	8-puzzle = state. TSP = path
5. Role of Knowledge	How much domain knowledge is required?	Medical diagnosis = extensive knowledge
6. Complexity	Time and space complexity of the search space	Chess has ~10^120 possible game states
7. Uncertainty	Does the problem involve incomplete or probabilistic information?	Weather prediction involves uncertainty
8. Dynamic Environment	Does the environment change while solving?	Autonomous driving in real traffic

Q3. Analyse 8-puzzle, Chess and Tower of Hanoi with respect to problem characteristics. [5 Marks]

Analysis of AI Problems Across All 8 Characteristics

Characteristic	8-Puzzle	Chess	Tower of Hanoi
1. Decomposable?	No — each tile move affects the entire board state	Partially — endgame can be analysed separately from opening	Yes — solve for n−1 disks first (recursive)
2. Steps Ignorable?	No — each move changes all tile positions; no undo	No — illegal or bad moves cannot be undone mid-game	No — must follow strict disc movement rules always
3. Absolute/Relative?	Absolute — one correct final arrangement of tiles	Relative — better strategy wins; no single “correct” game	Absolute — minimum 2^n −1 moves required exactly
4. State or Path?	State — only final solved state matters (goal test)	Path — the sequence of moves (strategy) determines outcome	Path — the ordered sequence of disk moves IS the solution
5. Role of Knowledge?	Low — heuristics like Manhattan distance and misplaced tiles	High — requires game theory, openings, tactics, endgames	Low — recursive algorithm is fully sufficient to solve
6. Complexity?	Moderate — 9! / 2 = 181,440 reachable states	Very High — ~10^120 possible game states (Shannon number)	Exponential — 2^n − 1 moves; O(2^n) time complexity
7. Uncertainty?	No — fully observable and deterministic environment	No — fully observable board (both players see all pieces)	No — fully deterministic, no random elements involved
8. Dynamic Environment?	No — static puzzle; only agent actions change state	Partially — opponent actions change environment dynamically	No — only one agent acts; environment is static always

Q4. Difference between ANN and BNN. [5 Marks]

ANN vs BNN — Architecture Diagram

```
BIOLOGICAL NEURON (BNN)          ARTIFICIAL NEURON (ANN)
+-----------------------+        +-------------------------+
| Dendrites (inputs)    |        | x1 ---w1--+             |
|   |||    |||    |||  |        | x2 ---w2--+--[Sigma]--f(z)-->Output
|    v      v      v   |        | x3 ---w3--+             |
|  [SOMA / Cell Body]   |        |           ^             |
|  (integration+fire)   |        |        bias (b)         |
|         |             |        |  z = Sum(wi*xi) + b     |
|       Axon            |        |  Output = f(z)          |
+-----------+----------+        +-------------------------+
            |
     Synapse to next neuron
```

Aspect	ANN (Artificial Neural Network)	BNN (Biological Neural Network)
Basic Unit	Artificial neuron (perceptron/node)	Biological neuron (nerve cell/soma)

Aspect	ANN (Artificial Neural Network)	BNN (Biological Neural Network)
Signal Type	Digital: mathematical operations (weighted sum + activation)	Electrochemical: ions, neurotransmitters, action potentials
Learning Rule	Backpropagation + gradient descent algorithms	Hebbian learning, synaptic plasticity (LTP / LTD)
Speed	Very fast: nanoseconds on GPU/TPU hardware	Slower (~100ms per recognition) but massively parallel
Scale	GPT-4 has ~1 trillion parameters (2023)	~86 billion neurons, ~100 trillion synapses in human brain
Energy Use	High: GPU consumes 250–400 Watts during training	Extremely efficient: entire brain runs on just ~20 Watts
Fault Tolerance	Vulnerable to single point of failure in network	Highly fault tolerant: lose neurons and still function
Adaptability	Requires full retraining for new tasks	Continuous lifelong learning via neuroplasticity
Parallelism	Limited by hardware (GPU cores)	Massively parallel: billions of neurons fire simultaneously
Application	Image classification, NLP, prediction, generation	All human/animal cognitive and motor functions

Q5. Explain different types of learning with examples. [4 Marks]

Types of Machine Learning

1. **Supervised Learning:** The model trains on labelled data (input + correct output). Learns to map inputs to outputs and predicts on new data. Examples: Spam detection (spam/not spam), House price prediction, Medical diagnosis (cancer/no cancer), MNIST digit classification.
2. **Unsupervised Learning:** Model trains on unlabelled data. Discovers hidden patterns, clusters, or structure without predefined outputs. Examples: Customer segmentation, Market basket analysis, Anomaly/fraud detection, Topic modelling in documents.
3. **Semi-Supervised Learning:** Combines a small amount of labelled data with a large amount of unlabelled data. Useful when labelling is expensive. Example: Google Photos — few labelled faces + thousands of unlabelled photos.
4. **Reinforcement Learning (RL):** An agent learns by interacting with an environment. It receives rewards for correct actions and penalties for wrong ones. Learns optimal policy through trial and error. Examples: AlphaGo, Self-driving cars, Robot navigation, Atari game agents.
5. **Deep Learning:** Uses multi-layered neural networks to learn complex hierarchical features from large datasets automatically. A subset of ML that powers modern AI. Examples: GPT-4 (text), DALL-E (images), AlphaFold (protein structure), Whisper (speech).

Q6. Difference between Supervised and Unsupervised Learning. [4 Marks]

Aspect	Supervised Learning	Unsupervised Learning
Data	Labelled: input + correct output (y) provided	Unlabelled: only input (X), no output labels
Goal	Learn a mapping f: X → y for prediction	Discover hidden patterns or natural structure in data
Output Type	Predefined class labels or continuous values	Clusters, associations, or reduced representations
Human Involvement	High — labels must be manually annotated	Low — no labelling needed; algorithm finds structure
Algorithms	Linear Regression, SVM, Decision Tree, Random Forest, Naive Bayes	K-Means, DBSCAN, PCA, Autoencoders, Apriori
Accuracy Measurement	Easy — compare predicted vs actual labels	Difficult — no ground truth for direct comparison
Real-world Example	Email spam classifier, Loan approval prediction	Customer segmentation, Document topic discovery
Evaluation Metrics	Accuracy, Precision, Recall, F1, RMSE, R^2	Silhouette Score, Inertia, Davies-Bouldin Index

Q7. Explain the elements of data science. [5 Marks]

Elements of Data Science

Data Science is an interdisciplinary field that combines statistics, programming, domain expertise, and machine learning to extract meaningful insights from large datasets. The six key elements are:

1. **Data Collection:** Gathering raw data from databases, APIs, web scraping, sensors, surveys, social media. Data types: Structured (relational tables), Semi-structured (JSON/XML), Unstructured (images, text, audio, video).
2. **Data Cleaning & Preprocessing:** Raw data is messy. This step handles missing values (imputation/removal), removes duplicates, fixes inconsistencies, standardises formats, and detects/handles outliers. This consumes 60–80% of a data scientist's time.
3. **Exploratory Data Analysis (EDA):** Investigate the data using statistical summaries and visualisations (histograms, scatter plots, box plots, heatmaps) to understand distributions, identify patterns, spot outliers, and form hypotheses before modelling.
4. **Machine Learning & Modelling:** Build predictive models using supervised (regression, classification), unsupervised (clustering), or deep learning techniques. Models are trained, validated, and optimised before deployment.
5. **Data Visualisation & Communication:** Present insights through charts, dashboards, and reports using Matplotlib, Seaborn, Tableau, Power BI. Goal: make complex findings understandable to non-technical stakeholders and decision-makers.
6. **Domain Knowledge:** Understanding the specific domain (healthcare, finance, retail) is essential for asking the right questions, correctly interpreting results, and making actionable, business-relevant recommendations.

Q8. Explain Data Visualisation techniques. [5 Marks]

Data Visualisation Techniques

Data Visualisation is the graphical representation of data and information using visual elements (charts, graphs, maps) to make patterns, trends, and outliers immediately recognisable to human perception.

Technique	Description	Best Used For	Python Tool
Bar Chart	Rectangular bars; height proportional to value	Comparing discrete categories (sales by region)	plt.bar()
Line Chart	Points connected by lines showing change over time	Time series data (stock prices, temperature)	plt.plot()
Histogram	Bars showing frequency distribution in bins	Understanding data distribution and skewness	plt.hist()
Pie Chart	Circle divided into proportional slices	Part-to-whole relationships (market share)	plt.pie()
Scatter Plot	Points on X-Y axes for two variables	Correlation analysis and outlier detection	plt.scatter()
Box Plot	Five-number summary with outlier markers	Statistical summaries and group comparison	sns.boxplot()
Heatmap	Colour-coded matrix of values	Correlation matrices, user behaviour analysis	sns.heatmap()
Violin Plot	Box plot + kernel density distribution	Probability distribution shape comparison	sns.violinplot()
Word Cloud	Word size reflects frequency in text	Text data and sentiment analysis	WordCloud lib

 Note: *Key Python libraries: Matplotlib (basic static plots), Seaborn (statistical), Plotly (interactive), Tableau/Power BI (business dashboards).*

▶ UNIT 2: EXPLORATORY DATA ANALYSIS (EDA)

Q9. What is Exploratory Data Analysis? [5 Marks]

Definition

Exploratory Data Analysis (EDA) is the process of investigating and summarising datasets to uncover their main characteristics, detect anomalies and outliers, test assumptions, discover patterns, and understand variable relationships — primarily using statistical graphics and visualisations, before applying any formal model. EDA was formalised by statistician John W. Tukey in the 1970s.

```
EDA Goal: Maximise insight into a dataset before modelling
```

EDA Pipeline (Visual Flow)

```
+-------------+    +---------------+    +-----------------+
| 1. Load     |    | 2. Overview   |    | 3. Handle       |
|   Data      |--->| .info()       |--->|   Missing       |
|             |    | .describe()   |    |   Values        |
+-------------+    | .shape        |    +-----------------+
                    +---------------+            |
+-------------+    +---------------+    +--------v--------+
| 6. Feature  |    | 5. Multivar   |    | 4. Univariate   |
|  Engineer   |<---| Analysis      |<---|   Analysis      |
|             |    | Heatmap,      |    |   Histogram,    |
+------v------+    | Pairplot      |    |   Boxplot       |
       |           +---------------+    +-----------------+
+------v------+
| 7. Modelling|
|   Ready!    |
+-------------+
```

Key Objectives of EDA

- Understand the structure and content of the dataset (rows, columns, data types, missing values).
- Identify outliers, anomalies, and data inconsistencies before modelling.
- Analyse distribution of individual variables (univariate analysis).
- Explore relationships between variables using scatter plots and correlation matrices.
- Generate and test hypotheses for further investigation or modelling.
- Guide feature engineering and feature selection decisions.

 Note: *EDA uses Python tools: Pandas for data manipulation, Matplotlib/Seaborn for visualisation, SciPy for statistical tests.*

Q10. Explain descriptive statistics. [4 Marks]

Descriptive Statistics

Descriptive Statistics is the branch of statistics that summarises and describes the main features of a dataset using numerical measures and visual representations. It describes only the data at hand and does NOT make predictions or inferences about a larger population.

A. Measures of Central Tendency (The 3 Ms)

- **Mean (μ):** Arithmetic average of all values.

```
μ = (x₁ + x₂ + ... + x□) / n = Σx / n     Example: {4,6,8,10} → Mean = 28/4 = 7
```

- **Median:** Middle value when data is sorted. Robust to outliers.

```
Odd n: Median = value at [(n+1)/2]th position Even n: Median = average of [n/2]th and [n/2+1]th values
```

- **Mode:** Most frequently occurring value. Can be unimodal, bimodal, or multimodal.

B. Measures of Dispersion (Spread)

- **Range:** Max − Min. Simple but sensitive to outliers.
- **Variance (σ^2):** Average of squared deviations from mean.

```
Population: σ² = Σ(xᵢ − μ)² / n     Sample: s² = Σ(xᵢ − x̄)² / (n−1)
```

- **Standard Deviation (σ):** Square root of variance. Same units as the data.

```
σ = √[Σ(xᵢ − μ)² / n]     Example: {2,4,6,8} → Mean=5, Var=5, SD≈2.24
```

- **IQR (Interquartile Range):** Q3 − Q1. Spread of middle 50% of data. Robust to outliers.

C. Measures of Shape

- **Skewness:** Measures asymmetry. Positive skew: tail to right (Mean > Median). Negative skew: tail to left (Mean < Median).
- **Kurtosis:** Measures peakedness. Leptokurtic (fat tails), Mesokurtic (normal), Platykurtic (thin tails).

Q11. Difference between Data and Histogram. [5 Marks]

Aspect	Data (Raw)	Histogram
Definition	Raw, unorganised collection of values, facts, or measurements	Graphical representation showing frequency distribution of numerical data grouped into bins
Form	Numbers, text, or other formats in tables, lists, or files	A bar chart where each bar represents count of values in a range
Readability	Difficult to identify patterns directly from raw numbers	Immediately reveals distribution shape, peaks, and outliers
Purpose	Input for computation and analysis algorithms	Summarise and visualise the distribution of numerical data
X-axis	Not applicable (just raw numbers or indices)	Value ranges (bins or intervals)
Y-axis	Not applicable	Frequency or count of data points in each bin
Bars/Gaps	N/A	No gaps between bars (continuous data unlike bar charts)
Example	Student marks: 45,72,63,81,55,90,67…	Bins: 40–50(2), 50–60(3), 60–70(5), 70–80(4)…

Parts of a Histogram

```
Frequency
   |
8  |     +-----+
7  |     |     |
6  |     |     +-----+
5  |     |     |     |
4  |+----+     |     +-----+
3  ||    |     |     |     +----+
2  ||    |     |     |     |    |
1  ||    |     |     |     |    |
   +-----+-----+-----+-----+-----+ Marks
   40-50  50-60 60-70 70-80 80-90
           <-- Bins (ranges) -->
```

Q12. Explain 3Ms. [3 Marks]

The 3 Ms — Measures of Central Tendency

The 3 M's are the three measures of central tendency: Mean, Median, and Mode. They indicate the centre or typical value in a dataset.

1. Mean (Arithmetic Average)

The mean sums all values and divides by the count. It is the most commonly used measure but is sensitive to outliers.

```
Mean (μ) = (x₁ + x₂ + … + x□) / n = Σx / n
```

Example: {10, 20, 30, 40, 50} → Mean = 150/5 = 30

Limitation: {10, 20, 30, 40, 500} → Mean = 120. Misleading because of the outlier 500.

2. Median (Middle Value)

The median is the middle value when data is sorted in ascending order. NOT affected by extreme values (outliers).

```
Odd n: Median = value at [(n+1)/2]th position Even n: Median = average of [n/2]th and [n/2+1]th values
```

Example (Odd): {3,5,7,9,11} → n=5, Median = 3rd value = 7

Example (Even): {2,4,6,8} → n=4, Median = (4+6)/2 = 5

3. Mode (Most Frequent Value)

```
Mode = Value with highest frequency in the dataset
```

Example: {4,5,5,6,7,5,8} → Mode = 5 (appears 3 times)

Bimodal Example: {2,3,3,4,5,5,6} → Mode = 3 and 5 (both appear twice)

Q13. Explain Measure of Dispersion. [5 Marks]

Measures of Dispersion

Measures of Dispersion describe how spread out or scattered data values are around the central value. While central tendency tells us WHERE the data is centred, dispersion tells us HOW MUCH the data varies.

1. Range

```
Range = Maximum Value − Minimum Value
```

Example: {3,7,2,9,5} → Range = 9−2 = 7. Limitation: Highly sensitive to outliers.

2. Variance (σ^2)

Variance measures the average of squared deviations from the mean.

```
Population Variance: σ² = Σ(xᵢ − μ)² / n Sample Variance: s² = Σ(xᵢ − x̄)²
/ (n−1)
```

Example: {2,4,6,8}, Mean=5 → Variance = $[(2-5)^2+(4-5)^2+(6-5)^2+(8-5)^2]/3 = [9+1+1+9]/3 = 20/3 \approx 6.67$

3. Standard Deviation (σ)

```
σ = √(Variance)      From above: SD = √6.67 ≈ 2.58
```

Standard deviation is expressed in the SAME UNIT as the original data (unlike variance which is squared).

4. Interquartile Range (IQR)

```
IQR = Q3 − Q1     (3rd Quartile minus 1st Quartile)
```

Steps: (1) Sort data (2) Find Q2 (Median) (3) Q1 = Median of lower half (4) Q3 = Median of upper half (5) IQR = Q3 − Q1

Example: {1,2,3,4,5,6,7,8,9,10} → Q1=3, Q3=8, IQR=5. Measures the middle 50% spread.

Note: *IQR is robust to outliers because it ignores the bottom 25% and top 25% of data values.*

Q14. Explain 5-Number Summary (Box-Plot Summary). [4 Marks]

The Five-Number Summary

The Five-Number Summary is a concise statistical description using five key values that capture the centre, spread, and extremes of a dataset. It forms the basis of the Box-and-Whisker Plot.

#	Name	Description
1	Minimum	Smallest value in the dataset (lower whisker tip)
2	Q1 (First Quartile)	25th percentile — 25% of values fall below this point
3	Q2 (Median)	50th percentile — divides dataset exactly in half
4	Q3 (Third Quartile)	75th percentile — 75% of values fall below this point
5	Maximum	Largest value in the dataset (upper whisker tip)

Box Plot Diagram

```
          Outlier                                   Outlier
            *                                         *
            |                                         |
   Min      |      Q1        Q2/Median     Q3       Max
   |--------|-------[========|============]--------|
   |        |        |        |            |        |
   10      13       17       22           28       35
           <--IQR = Q3 - Q1 = 11 -------->
   Whisker: extends to Min/Max (or 1.5*IQR)
   Outlier: point beyond Q1-1.5*IQR or Q3+1.5*IQR
```

Worked Example

Dataset (sorted): {10, 12, 14, 15, 18, 20, 22, 25, 30} (n=9)

1. n = 9 (odd) → Q2 (Median) = 5th value = 18
2. Lower half: {10,12,14,15} → Q1 = (12+14)/2 = 13
3. Upper half: {20,22,25,30} → Q3 = (22+25)/2 = 23.5
4. Five-Number Summary: Min=10, Q1=13, Median=18, Q3=23.5, Max=30
5. IQR = Q3 − Q1 = 23.5 − 13 = 10.5

```
  Outlier Boundaries: Lower = Q1 - 1.5×IQR = 13-15.75 = -2.75  |  Upper = Q3
+ 1.5×IQR = 23.5+15.75 = 39.25
```

▶ UNIT 3: INFERENTIAL STATISTICS

Q15. Explain the concept of probability and its types. [5 Marks]

Probability

Probability is the mathematical measure of the likelihood that a specific event will occur. It is a value between 0 (impossible) and 1 (certain). Probability provides a formal framework for quantifying uncertainty in experiments and decisions.

```
  P(A) = Number of Favourable Outcomes / Total Number of Outcomes     [0 ≤ P(A) ≤ 1]
```

Example: Rolling a fair six-sided die: P(getting a 4) = 1/6 ≈ 0.167 (16.7% chance)

Types of Probability

Type	Description	Formula / Basis	Example
Classical / Theoretical	Based on equally likely outcomes using counting. Known before experiment.	P(A) = favourable / total	P(Head) on fair coin = 1/2
Empirical / Statistical	Based on relative frequency of past observations from experiments.	P(A) = observed freq / total trials	Rain on 30 of 100 days → P=0.30
Subjective	Based on personal judgment or intuition, not calculation.	Expert opinion	Doctor: '70% chance patient recovers'
Conditional	P(A|B): probability of A given B has already occurred.	P(A|B) = P(A∩B) / P(B)	P(disease | +ve test result)
Joint	P(A∩B): both events A and B occur simultaneously.	P(A∩B) = P(A)×P(B|A)	P(rolling 4 AND getting heads)
Marginal	Probability of a single event regardless of other events.	Sum across all joint outcomes	P(rolling a 4) = 1/6

Key Probability Rules

```
Addition Rule:        P(A∪B) = P(A) + P(B) − P(A∩B)
Multiplication Rule:  P(A∩B) = P(A) × P(B|A)
Complement Rule:      P(A') = 1 − P(A)
```

Q16. Difference between Descriptive and Inferential Statistics. [3 Marks]

Aspect	Descriptive Statistics	Inferential Statistics
Definition	Summarises and describes features of a KNOWN dataset	Makes predictions or inferences about a POPULATION based on a sample
Purpose	Organise, summarise, and present data meaningfully	Draw conclusions about a larger population from sample data

Aspect	Descriptive Statistics	Inferential Statistics
Data Scope	Uses the entire available dataset	Uses a subset (sample) to generalise to the whole population
Conclusion	No predictions — only describes what the data currently shows	Makes predictions, estimates, and hypothesis test decisions
Output	Tables, charts, mean, median, mode, variance, SD	Confidence intervals, p-values, hypothesis test results
Uncertainty	No uncertainty (describes existing data exactly)	Always involves uncertainty (sampling error, confidence levels)
Techniques	Mean, Median, Mode, Range, SD, IQR, Histogram, Box Plot	T-test, ANOVA, Chi-square, Regression, Bayes' theorem
Example	"The average exam score in this class is 72%"	"From 100 students, we estimate 70% of all students pass"

Q17. Explain types of Inferential Statistics. [5 Marks]

Types of Inferential Statistics

Inferential Statistics uses sample data to make generalisations, predictions, or decisions about a larger population. The main types are:

1. **Hypothesis Testing:** Tests whether a claim about a population is statistically supported. Steps: Define H_0 (null) and H_1 (alternative) → Collect sample → Calculate test statistic → Compare with critical value → Accept or reject H_0 . Example: Testing if a new drug reduces blood pressure significantly.
2. **Confidence Intervals (CI):** A range of values likely to contain the true population parameter with a specified confidence level (typically 95% or 99%). `CI = x¯ ± z×(σ/√n)  where z=1.96 for 95%`
3. **Regression Analysis:** Estimates relationships between variables. Linear regression predicts a continuous outcome; logistic regression predicts class probability. Used for forecasting and trend analysis.
4. **Analysis of Variance (ANOVA):** Tests whether means of three or more groups are significantly different from each other. F-statistic compares between-group variance to within-group variance. Example: Test if exam scores differ across three teaching methods.
5. **Chi-Square Test (χ^2):** Tests association between two categorical variables. Compares observed frequencies to expected frequencies. Example: Is there a relationship between gender and preference for a product?
6. **Bayesian Inference:** Uses Bayes' theorem to update probability estimates as new evidence arrives. Combines prior beliefs with observed data to get a posterior distribution. Central to Naive Bayes classifier and probabilistic AI.

Q18. Explain Random Variables with its types. [3 Marks]

Random Variables

A Random Variable is a variable whose value is determined by the outcome of a random experiment. It assigns a numerical value to each outcome in the sample space. Random variables are always measurable and represented by real numbers.

Example: Tossing two fair coins. Let X = number of heads. X can take values {0, 1, 2}.

Feature	Discrete Random Variable	Continuous Random Variable
Definition	Takes countable, finite or countably infinite distinct values	Takes any real value within a continuous range or interval
Values	Whole numbers or specific countable values only	Any real number within a range (including all decimals)
Probability Function	PMF: $P(X=x)$ gives exact probabilities	PDF: $f(x)$ gives density; $P(a \le X \le b) = \int f(x)dx$
Sum/Integral Property	$\Sigma P(x_i) = 1$ (sum of all probabilities equals 1)	$\int f(x)dx = 1$ (total area under curve equals 1)
P(X = exact value)	Can be non-zero (has a specific value)	Always 0 for exact point; ranges have non-zero probability
Graph Type	Bar graph (separate, isolated bars)	Smooth, continuous bell/skewed curve
Examples	Dice roll (1–6), Number of heads (0,1,2,3), Students absent per day	Height, weight, temperature, time, distance, voltage

```
Mean: E(X) = Σ xᵢ·P(X=xᵢ)   [discrete]   OR   ∫ x·f(x)dx   [continuous]

Variance: Var(X) = E(X²) − [E(X)]²
```

Q19. Explain the Central Limit Theorem with its rules. [5 Marks]

Central Limit Theorem (CLT)

The Central Limit Theorem (CLT) is one of the most fundamental theorems in statistics. It states that regardless of the shape of the original population distribution, the sampling distribution of the sample mean approaches a Normal (Gaussian) distribution as the sample size (n) increases, provided samples are independent and identically distributed.

```
If X̄ is sample mean: X̄ ~ N(μ, σ²/n) as n → ∞
```

CLT Visualisation

```
Original (any shape)        Sample Means (n=5)        Sample Means (n=30)
+------------------+        +------------------+      +------------------+
|  ___             |        |        _         |      |       ___        |
| |   |            |        |       / \        |      |      /   \       |
| |   |__          |        |      /   \       |      |     /     \      |
| |      |__       |        |     /     \      |      |    /       \     |
| |         |__    |        |    /       \     |      |   /         \    |
+--+--+--+--+---+  |        +---------+------+  |      +--------+---------+
Skewed distribution           Becoming normal            Nearly perfect
                                                         normal curve!
```

Rules / Conditions for CLT

Rule	Description
1. Random Sampling	Samples must be randomly and independently selected to represent the population
2. Independence	Each observation must be independent — one value should not influence another
3. Sample Size ≥ 30	A sample size ≥ 30 is generally sufficient for CLT to apply (rule of thumb)
4. Finite Mean & Variance	Population must have a defined (finite) mean and variance for CLT to hold
5. Identical Distribution	All samples should come from the same population distribution

Worked Example

Population: mean μ=70 kg, standard deviation σ=15 kg. Researcher samples n=50 men.

```
Mean of sampling distribution = μ = 70 kg  (same as population mean)
Standard Error = σ/√n = 15/√50 = 15/7.07 ≈ 2.12 kg
```

Conclusion: Distribution of sample means will be Normal(70, 2.12^2) regardless of the original shape of the weight distribution.

Q20. Explain sampling distribution with its types. [4 Marks]

Sampling Distribution

A Sampling Distribution is the probability distribution of a statistic (e.g., sample mean, sample proportion) computed from ALL possible random samples of a fixed size drawn from a population. It describes how the statistic would vary from sample to sample if you repeated the sampling process infinitely many times.

```
SE (Standard Error of Mean) = σ/√n     [Smaller SE → More precise estimates]
```

Type	Description	Formula / Properties
Sampling Dist. of Mean	Distribution of $\bar{X}$ from all possible samples of size n	Mean=μ, SE=σ/√n; becomes Normal for n≥30 (by CLT)
Sampling Dist. of Proportion	Distribution of sample proportion $\hat{p}$ for categorical data	Mean=p, SE=√[p(1-p)/n]
t-Distribution	Used when population SD unknown and n<30	Bell-shaped with heavier tails; converges to Normal as n→∞
Chi-Square (χ^2) Distribution	Distribution of variance statistic; goodness-of-fit	Right-skewed; df = n−1; always positive
F-Distribution	Ratio of two chi-square distributions; used in ANOVA	Right-skewed; defined by two degrees of freedom (df_1 , df_2)

Q21. Explain Cross Validation with its types. [8 Marks]

Cross Validation

Cross Validation (CV) is a model evaluation technique that tests how well a machine learning model generalises to an independent dataset. It detects overfitting (model memorises training data but fails on new data) and provides a reliable, unbiased estimate of model performance on unseen data.

K-Fold Cross Validation Diagram

```
Dataset split into K=5 folds:
+-------+-------+-------+-------+-------+
|Fold 1 |Fold 2 |Fold 3 |Fold 4 |Fold 5 |
+-------+-------+-------+-------+-------+
Iter 1: [TEST  ][TRAIN  |TRAIN  |TRAIN  |TRAIN ]
Iter 2: [TRAIN ][TEST   |TRAIN  |TRAIN  |TRAIN ]
Iter 3: [TRAIN |TRAIN  ][TEST  ][TRAIN  |TRAIN ]
Iter 4: [TRAIN |TRAIN  |TRAIN  ][TEST  ][TRAIN ]
Iter 5: [TRAIN |TRAIN  |TRAIN  |TRAIN  ][TEST  ]
Final Score = Average of 5 iteration scores
```

Types of Cross Validation

1. **K-Fold Cross Validation (Most Common):** Dataset split into K equal folds. Model trained on K−1 folds, tested on the remaining 1 fold. Repeated K times. Final score = average of all K scores. K=5 or K=10 is typical.

$$\text{Final Score} = (\text{Score}_1 + \text{Score}_2 + \ldots + \text{Score}_K) / K$$

2. **Stratified K-Fold:** Variant of K-Fold that ensures each fold has the same class distribution as the original dataset. Critical for imbalanced datasets (e.g., 90% negative, 10% positive). Use case: Medical diagnosis where disease cases are rare.
3. **Leave-One-Out CV (LOOCV):** Special case where K=n (total samples). Each iteration uses ONE sample as test set and all others for training. Most accurate but computationally very expensive for large datasets. Advantage: Uses maximum training data at each step.
4. **Hold-Out Validation:** Simplest form: split data once into train (70–80%) and test (20–30%) sets. Fast but unreliable for small datasets since results depend heavily on which samples land in each set.
5. **Time Series Cross Validation (Walk Forward):** For temporal data where order must be preserved. Training always uses past data; testing uses future data. Never uses future data to predict the past (prevents data leakage in financial and weather models).

Q22. Explain Bayesian theorem with its importance. [5 Marks]

Bayes' Theorem

Bayes' Theorem, developed by Reverend Thomas Bayes (published posthumously in 1763), describes how to update the probability of a hypothesis based on new evidence. It is the mathematical foundation of probabilistic reasoning and is central to modern AI and machine learning.

$$P(A|B) = [\ P(B|A) \times P(A)\ ] / P(B)$$

Term	Name	Meaning
P(A|B)	Posterior Probability	Updated probability of A AFTER seeing evidence B — what we want to find

Term	Name	Meaning
P(B|A)	Likelihood	Probability of observing evidence B GIVEN that hypothesis A is true
P(A)	Prior Probability	Initial probability of A BEFORE seeing any new evidence
P(B)	Marginal / Evidence	Total probability of evidence B occurring across ALL hypotheses

Worked Numerical Example

Problem: A construction job must be completed on time. With rain: P(complete|rain)=0.44. Without rain: P(complete|no rain)=0.95. Probability of rain = 0.45. Find P(job completed on time).

```
P(E) = P(Rain)×P(Complete|Rain) + P(No Rain)×P(Complete|No Rain)
P(E) = 0.45×0.44 + 0.55×0.95 = 0.198 + 0.5225 = 0.7205 (72.05%)
```

Importance / Applications of Bayes' Theorem

- **Spam Filtering:** Calculates P(spam | email_words) using word frequency statistics from known spam/ham training emails.
- **Medical Diagnosis:** P(disease | positive test) — updates disease probability considering test sensitivity and disease prevalence.
- **Naive Bayes Classifier:** A popular ML algorithm for text classification built entirely on Bayes' theorem with naive independence assumption.
- **Risk Assessment:** Insurance companies use Bayes to update risk estimates as new customer behaviour data becomes available.
- **Bayesian Networks:** Graphical models that encode probabilistic relationships among variables using conditional probability tables based on Bayes' theorem.

▶ UNIT 4: SUPERVISED LEARNING

Q23. What is Supervised Learning? Explain the problems in Supervised Learning. [5 Marks]

Supervised Learning

Supervised Learning is a type of machine learning where the model is trained on a labelled dataset containing both input features (X) and the corresponding correct output labels (y). The algorithm learns to map inputs to outputs based on example pairs, then predicts outputs for new, unseen inputs.

```
  Goal: Learn function f such that f(X) ≈ y   for all (X, y) in training
data
```

Supervised Learning has two main problem types:

- **Classification:** Predicts a discrete class label. Example: Email is Spam (1) or Not Spam (0).
- **Regression:** Predicts a continuous numerical value. Example: Predicting house price from area and location.

Problems in Supervised Learning

Problem	Description	Solution
Overfitting	Model memorises training data (noise included) and fails on new data. High train accuracy, low test accuracy.	Regularisation (L1/L2), Dropout, Cross Validation, Reduce model complexity
Underfitting	Model too simple — fails to capture patterns even in training data. Low accuracy on both train and test.	Increase model complexity, Add more features, Train for more epochs
Bias-Variance Tradeoff	High bias = underfitting. High variance = overfitting. Hard to minimise both simultaneously.	Find optimal complexity via cross-validation and learning curves
Insufficient Labelled Data	Labelling is expensive, time-consuming, and requires domain experts.	Data augmentation, Transfer learning, Semi-supervised learning
Class Imbalance	95% non-fraud / 5% fraud. Model becomes biased towards the majority class.	SMOTE oversampling, class weights, ensemble methods, ROC-AUC evaluation
Feature Selection	Irrelevant or redundant features reduce accuracy and increase computation.	Feature importance, PCA, Lasso regularisation, domain knowledge
Concept Drift	Real-world data distribution changes over time making the model outdated.	Continuous monitoring and periodic retraining with new data

Q24. Difference between Classification and Regression. [4 Marks]

Aspect	Classification	Regression
Definition	Predicts discrete class labels or categories	Predicts continuous numerical values

Aspect	Classification	Regression
Output	Categorical (finite set of classes)	Continuous (any real number)
Question Answered	"Which category does this belong to?"	"How much / How many?"
Decision Boundary	Finds a boundary that separates different classes	Fits a curve/line to minimise prediction error
Evaluation Metrics	Accuracy, Precision, Recall, F1-Score, AUC-ROC	MSE, RMSE, MAE, R^2 (R-squared)
Algorithms	Decision Tree, SVM, Naive Bayes, KNN, Logistic Regression	Linear Regression, Polynomial Regression, Ridge, Lasso
Output Format	Class label (e.g., "Cat", "Dog", 0, 1, "Spam")	Number (e.g., 45000, 98.6, 0.75)
Examples	Email spam detection, Disease diagnosis, Image recognition	House price, Stock price, Temperature forecasting

Q25. Explain Linear Regression. [3 Marks]

Linear Regression

Linear Regression is a supervised ML algorithm that models the relationship between a dependent variable (y) and one or more independent variables (X) by fitting a straight line through the data. It is the simplest and most widely used regression technique.

```
Simple:   y = β0 + β1X + ε
Multiple: y = β0 + β1X1 + β2X2 + … + β□X□ + ε
```

Where: y=predicted output, β_0 =y-intercept, β_1 =slope, X=input feature, ε=error term

Linear Regression Diagram

```
 y |
   |         *              Best Fit Line: y = b0 + b1*x
   |      * /  *            Residual (error) = actual - predicted
   |    */        *         Least Squares: Minimise Sum(y_i - y^_i)^2
   |  *  /   *
   |* /  *                  Slope b1 = Cov(X,Y) / Var(X)
   |/________*_________ x
  b0  b0+b1
  (intercept when X=0)
```

Types of Linear Regression

Type	Description	Example
Simple Linear	One independent variable (X) and one dependent (y)	Predicting salary from years of experience
Multiple Linear	Two or more independent variables	House price from size, location, and age
Ridge Regression	Linear + L2 regularisation ($\lambda\Sigma\beta^2$). Handles multicollinearity.	When features are correlated
Lasso Regression	Linear + L1 regularisation ($\lambda\Sigma\lvert\beta\rvert$). Performs feature selection.	High-dimensional sparse data

```
Ordinary Least Squares (OLS): Minimise Σ(yᵢ − ŷᵢ)²  where ŷᵢ = β₀ + β₁xᵢ
```

Q26. Explain Logistic Regression. [5 Marks]

Logistic Regression

Logistic Regression is a supervised ML algorithm used for CLASSIFICATION problems. Despite having "regression" in its name, it predicts the PROBABILITY of a binary outcome (0 or 1). It uses the Sigmoid (Logistic) function to transform linear output into a probability between 0 and 1.

```
P(y=1|x) = 1 / (1 + e^{-(β₀+β₁x)})   [Sigmoid Function]
```

Sigmoid Function Diagram

```
P(y=1)
1.0  |                    .......--------
     |                ....
0.75 |              ..
     |            .
0.5  |..........* <-- Decision boundary z=0
     |        .
0.25 |      ..
     |   ...
0.0  |....
     +---+---+---+---+---+---+---+---+
    -4  -3  -2  -1   0   1   2   3   4   z = b0 + b1*x
Rule: If P >= 0.5 -> Class 1 (positive)  |  If P < 0.5 -> Class 0 (negative)
```

Type	Description	Example
Binomial Logistic Regression	Two possible outcome categories. Most common type.	Spam vs Not Spam; Disease vs No Disease
Multinomial Logistic Regression	Three or more unordered outcome categories.	Cat/Dog/Bird; Sentiment: +/−/Neutral
Ordinal Logistic Regression	Three or more ordered/ranked outcome categories.	Rating: 1-star/2-star/3-star; Severity: Low/Med/High

Key Properties

- Output always between 0 and 1 — can be directly interpreted as probability.
- S-shaped sigmoid curve approaches 0 for very negative z, 1 for very positive z.
- Decision boundary at P=0.5 corresponds to z=0 (where $\beta_0 + \beta_1 x = 0$).
- Uses Log Loss (Binary Cross-Entropy) as loss function, not MSE.

Q27. Explain Polynomial Regression. [3 Marks]

Polynomial Regression

Polynomial Regression extends linear regression by modelling the relationship between X and y as an nth-degree polynomial. Used when data exhibits a CURVED (non-linear) relationship that a straight line cannot capture.

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \ldots + \beta_n x^n + \varepsilon$$

Although it produces a curved fit, it is still called "linear regression" because the model is LINEAR IN ITS PARAMETERS (β values). The polynomial terms (x^2, x^3) are just feature transformations.

```
Linear (Degree 1):     y = β₀ + β₁x
Quadratic (Degree 2): y = β₀ + β₁x + β₂x²
Cubic (Degree 3):      y = β₀ + β₁x + β₂x² + β₃x³
```

Steps in Polynomial Regression

1. Collect dataset and visualise to confirm non-linear (curved) trend in the data.
2. Transform input features: create polynomial features (x, x^2, x^3, … up to degree n).
3. Apply Linear Regression on the transformed (expanded) polynomial feature matrix.
4. Train the model using Ordinary Least Squares to find optimal β coefficients.
5. Evaluate model performance using RMSE, R^2, and residual plots to check fit.

Note: *High degree polynomials risk OVERFITTING. Use cross-validation to select the optimal degree.*

Q28. Explain Decision Tree. [5 Marks]

Decision Tree

A Decision Tree is a supervised ML algorithm that creates a tree-like model of decisions. Each internal node represents a test on a feature, each branch represents the outcome, and each leaf node holds the final prediction. It is interpretable, requires no feature scaling, and handles both numerical and categorical data.

Decision Tree Diagram

```
                    +-------------------+
                    |   Root Node:      |
                    |   Attendance?     |   <-- Most important feature
                    +---+----------+----+
                        |          |
                    [HIGH]        [LOW]
                        |          |
            +-----------+    +-----+----------+
            |                |   Study Hours? |
        [PASS]               +---+--------+---+
       Leaf Node           [Enough]     [Less]
                              |           |
                           [PASS]       [FAIL]
                          Leaf Node    Leaf Node
```

Attribute Selection: Information Gain & Gini Index

```
Entropy(S) = -Σ pᵢ × log₂(pᵢ)      [0 = pure, 1 = maximum impurity]
Information Gain = Entropy(Parent) - [Weighted Average Entropy(Children)]
Gini Index = 1 - Σ(pᵢ)²      [0 = pure, 0.5 = maximum impurity for 2 classes]
```

Advantages	Disadvantages
Easy to understand and visualise (white-box model)	Can overfit if tree is too deep (no pruning)

Advantages	Disadvantages
No feature scaling / normalisation needed	Unstable — small data changes can completely alter tree
Handles both numerical and categorical features	Biased towards features with more categories (info gain)
Captures non-linear decision boundaries naturally	Less accurate than ensemble methods (Random Forest, XGBoost)
Can handle missing data with modifications	Computationally expensive for large feature sets

Q29. Explain Random Forest. [5 Marks]

Random Forest

Random Forest is an ensemble learning algorithm that builds multiple decision trees and combines their predictions. For classification: majority voting. For regression: average of all tree predictions. Ensemble approach overcomes the weaknesses of a single decision tree.

Random Forest Architecture

```
Original Dataset
     |
Bootstrap Sampling (with replacement)
+--------+  +--------+  +--------+  +--------+
|Sample 1|  |Sample 2|  |Sample 3|  |Sample N|
+---+----+  +---+----+  +---+----+  +---+----+
    |           |           |           |
+-------+  +-------+  +-------+  +-------+
|Tree 1 |  |Tree 2 |  |Tree 3 |  |Tree N |    (each uses random
| [Y=1] |  | [Y=0] |  | [Y=1] |  | [Y=1] |     subset of features)
+-------+  +-------+  +-------+  +-------+
                  |
         AGGREGATION (Voting / Average)
                  |
           Final Prediction
```

Working of Random Forest — Step by Step

1. **Bootstrap Sampling:** Randomly select multiple subsets of training data WITH replacement (each subset is ~63% unique). Each subset builds one tree.
2. **Random Feature Selection:** At each node split, consider only a random subset of features (√p for classification, p/3 for regression, where p=total features). Ensures diversity among trees.
3. **Build Individual Trees:** Each decision tree is fully grown using its random data and feature subsets. Each tree makes slightly different decisions.
4. **Aggregate Predictions:** Classification → Majority Voting (most common class wins). Regression → Mean of all tree predictions.

Advantages	Disadvantages
Very high accuracy even on complex datasets	Slower training and prediction than single trees
Reduces overfitting through averaging (bagging)	Hard to interpret ("black box") unlike single trees
Handles missing values without imputation	Requires more memory to store multiple trees

Advantages	Disadvantages
No feature scaling required	Slow prediction for real-time applications with many trees
Provides feature importance scores automatically	Not ideal for very high-dimensional sparse data

Q30. Explain Naive Bayes. [3 Marks]

Naive Bayes Classifier

Naive Bayes is a probabilistic classification algorithm based on Bayes' theorem. It is called "Naive" because it makes a strong assumption that ALL FEATURES ARE CONDITIONALLY INDEPENDENT given the class label — rarely true in reality but works surprisingly well in text classification.

```
P(C|X) = P(X|C) × P(C) / P(X)
```

With naive independence: $P(X|C) = P(x_1|C) \times P(x_2|C) \times \ldots \times P(x_{\square}|C)$

Type	Assumption	Best Used For
Gaussian Naive Bayes	Continuous features follow a Normal (Gaussian) distribution	Iris classification, real-valued features like height/weight
Multinomial Naive Bayes	Features represent discrete counts or frequencies	Text classification, document categorisation, word count models
Bernoulli Naive Bayes	Features are binary (0 or 1) — presence or absence	Spam detection based on word presence/absence in emails

□ **Note:** *Zero-Probability Problem: If a feature value never appears in training for a class, P=0 kills the whole product. Solution: Laplace Smoothing adds 1 to all frequency counts.*

Q31. Explain SVM (Support Vector Machine). [4 Marks]

Support Vector Machine (SVM)

SVM is a supervised ML algorithm that finds the OPTIMAL HYPERPLANE maximising the margin between classes. The data points closest to the hyperplane from each class are called Support Vectors — they are the only points that define the decision boundary.

```
Hyperplane equation: w·x + b = 0      |      Margin = 2 / ||w||   (SVM maximises this)
```

SVM Diagram

```
Class A (+)                    Class B (-)
*                                          o
*   *       MARGIN       /      o   o
*     *     +-------+   /  o   o
         *  |SUPPORT|  /  o
*           |VECTORS| /   o    o
        *   +-------+/  o
*             /   Support Vectors ---> closest points to hyperplane
             /  <-- Optimal Hyperplane (w.x + b = 0)
Margin = 2/||w||  <-- SVM maximises this gap
```

Key Concepts

Concept	Description
Hyperplane	Decision boundary (line in 2D, plane in 3D, hyperplane in nD). Equation: $w^T x + b = 0$
Support Vectors	Closest data points to hyperplane from each class. ONLY these points determine the boundary.
Hard Margin SVM	Perfect separation without any misclassification — works only when data is linearly separable
Soft Margin SVM (C parameter)	Allows some misclassification using slack variables (ξ). High C: fewer misclassifications, smaller margin. Low C: larger margin, more misclassifications.
Kernel Trick	Maps data to higher dimensions where linear separation becomes possible without explicit computation

Common Kernel Functions

Kernel	Formula	Use Case
Linear	$K(x,y) = x^T y$	Linearly separable data, high-dimensional text classification
Polynomial	$K(x,y) = (x^T y + c)^d$	Non-linear boundaries, image classification
RBF (Gaussian)	$K(x,y) = \exp(-\gamma\|\|x-y\|\|^2)$	General purpose, most commonly used kernel for non-linear data
Sigmoid	$K(x,y) = \tanh(\alpha x^T y + c)$	Neural network-like classification behaviour

▶ UNIT 5: UNSUPERVISED LEARNING

Q32. Explain Unsupervised Learning. Explain the problems. [3 Marks]

Unsupervised Learning

Unsupervised Learning is ML where the algorithm trains on UNLABELLED data (no predefined correct output labels). It discovers hidden patterns, natural groupings, or intrinsic structure entirely on its own, without human supervision or guidance.

```
Training data = {X} only — no labels (y) provided
```

Application	Example
Customer Segmentation	Group customers by purchasing behaviour for targeted marketing campaigns
Anomaly Detection	Identify unusual credit card transactions (fraud) or network intrusions
Dimensionality Reduction	PCA reduces 1000-dimensional image data to 50 dimensions for storage/speed
Recommender Systems	Netflix groups users by viewing patterns to recommend similar content
Document Clustering	Group news articles by topic without any manual category labels

Problems in Unsupervised Learning

Problem	Description	Solution
No Ground Truth	No labels to verify correctness. Hard to know if clusters are meaningful.	Silhouette Score, Inertia, domain expert validation
Choosing K	K-Means requires specifying number of clusters in advance — not always known.	Elbow Method, Silhouette Analysis, Gap Statistic
High Dimensionality	Many features make distance metrics unreliable (Curse of Dimensionality).	Dimensionality reduction (PCA, t-SNE) before clustering
Scale Sensitivity	K-Means uses Euclidean distance; features with large scales dominate.	Normalise/standardise all features before clustering
Interpretability	Clusters found may not have clear business meaning.	Domain knowledge required for cluster interpretation

Q33. Explain K-Means Clustering. [4 Marks]

K-Means Algorithm

K-Means partitions a dataset into K distinct, non-overlapping clusters where each data point belongs to the cluster with the nearest centroid (mean). Iteratively assigns points and updates centroids until convergence.

```
Objective: Minimise WCSS = Σ□ Σᵢ∈C□ ||xᵢ − μ□||  [Within-Cluster Sum of Squares]
```

K-Means Iteration Diagram

```
STEP 0: Init    STEP 1: Assign  STEP 2: Update   STEP 3: Converged
+----------+    +----------+    +----------+     +----------+
|  . . X   |    |  . . *   |    |  . . *   |     |  .+. |   |
|. . . . . |    |. . . . . |    | . . . .  |     | . . .|. .|
|. .   X ..|    |. . * . ..|    | .  *  .  |     | . .* | ..|
| .  . . . |    | .  . . . |    |  .  .  . |     |  . * | . |
+----------+    +----------+    +----------+     +------+---+
X = initial     * = centroid     * moved to      | = final boundary
  centroids     colour=cluster    new mean         * = final centroid
```

Step-by-Step Algorithm

1. **Choose K:** Decide the number of clusters. Use the Elbow Method to find the optimal K.
2. **Initialise Centroids:** Randomly select K data points as initial cluster centroids (or use K-Means++ for better initialisation).
3. **Assignment Step:** Assign each data point to the nearest centroid using Euclidean distance: $\text{dist}(x, \mu_C) = \sqrt{\Sigma(x_i - \mu_{C,i})^2}$
4. **Update Step:** Recalculate each centroid as the mean of all assigned points: $\mu_C = (1/|C_C|) \times \Sigma_{i \in C_C} x_i$
5. **Convergence Check:** Repeat steps 3–4 until centroids no longer move significantly (or max iterations reached).

Elbow Method for Optimal K

```
WCSS
  |
  | *
  |   *
  |     *
  |       *  <-- Elbow point (optimal K=3)
  |          *  *  *  *
  +---+---+---+---+---+--- K
      1   2   3   4   5
WCSS drops sharply then plateaus at elbow -> that K is optimal
```

Advantages	Disadvantages
Simple, easy to implement and understand	Sensitive to initial centroid placement (may converge to local optima)
Computationally efficient: O(nKI)	Must specify K in advance — not always known
Scales well to large datasets	Assumes spherical, equal-sized, equal-density clusters
Guaranteed to converge	Cannot handle non-convex cluster shapes (rings, crescents)

Q34. Explain PCA (Principal Component Analysis). [3 Marks]

PCA — Dimensionality Reduction

PCA transforms a high-dimensional dataset with correlated features into a smaller set of UNCORRELATED features called Principal Components (PCs). Each PC is a linear combination of original features. The first PC explains the most variance, the second the next most, and so on.

PCA Visual Diagram

```
Original 3D Space          After PCA (2D projection)
    z|                          PC2|
     | * *                          |   * *
     |*   *                      * *  |  *   *
     |  *   *                   *   * | *   * *
     +--------x                  +------------ PC1
    /  *  *                     (PC1 = max variance direction)
   y                            (PC2 = 2nd max, perpendicular to PC1)
                                 Data compressed from 3D to 2D!
```

PCA Algorithm — Step by Step

1. **Standardise:** Subtract mean and divide by SD for each feature: $z = (x - \mu) / \sigma$. Ensures all features are on the same scale.
2. **Covariance Matrix:** Compute Cov(X) to capture relationships between all feature pairs.
3. **Eigen Decomposition:** Compute eigenvalues (variance explained) and eigenvectors (PC directions) of the covariance matrix.
4. **Sort by Eigenvalue:** Sort eigenvectors in descending order of eigenvalue. Eigenvector with largest eigenvalue = PC1.
5. **Select Top K Components:** Choose K eigenvectors that explain sufficient cumulative variance (e.g., 95%). Reduces n features to K components.
6. **Project Data:** Transform original data onto K PCs: $Z = X \times W$ (W = matrix of selected eigenvectors).

Applications

- **Visualisation:** Reduce 50-dimensional data to 2D/3D for scatter plotting and cluster exploration.
- **Noise Reduction:** Drop PCs with very low eigenvalues (low variance = likely noise).
- **Speed up ML:** Fewer features → faster training. Use PCA before applying SVM or Logistic Regression.
- **Face Recognition:** Eigenfaces method: PCA on face images to find most important facial variation components.

Q35. Explain different Libraries of Python for ML. [5 Marks]

Python Libraries for Machine Learning & Data Science

Library	Category	Key Functions / Features
NumPy	Numerical Computing	N-dimensional arrays, broadcasting, matrix ops, linear algebra, random module
Pandas	Data Manipulation	DataFrame, Series, read_csv(), groupby(), merge(), fillna(), describe(), pivot_table()
Matplotlib	Visualisation (Basic)	plt.plot(), plt.bar(), plt.hist(), plt.scatter(), plt.subplots(), plt.show()
Seaborn	Statistical Viz	Built on Matplotlib: heatmap(), pairplot(), boxplot(), violinplot(), distplot()
Scikit-learn	Machine Learning	LinearRegression, DecisionTreeClassifier, RandomForest, SVM, KMeans, PCA, train_test_split, cross_val_score, GridSearchCV

Library	Category	Key Functions / Features
TensorFlow / Keras	Deep Learning	Build neural networks with Sequential() API, model.fit(), model.predict(), GPU acceleration
PyTorch	Deep Learning (Research)	Dynamic computation graphs, autograd, tensors, preferred by researchers for flexibility
SciPy	Scientific Computing	Statistical tests (t-test, ANOVA, chi-square), optimisation, signal processing, integration
NLTK / SpaCy	NLP	Tokenisation, stemming, lemmatisation, POS tagging, Named Entity Recognition (NER)
XGBoost / LightGBM	Gradient Boosting	High-performance boosted trees; winner of many Kaggle competitions

 Note: *Typical ML workflow: Pandas (data) → Seaborn (EDA) → Scikit-learn (models) → Matplotlib (results). For deep learning: NumPy + TensorFlow/Keras or PyTorch.*

▶ UNIT 6: NEURAL NETWORKS

Q36. Explain Neural Networks. List applications, advantages, and limitations. [5 Marks]

Neural Networks

A Neural Network (NN) is a computational model inspired by the structure of the biological brain. It consists of interconnected layers of artificial neurons (nodes) that process information through weighted connections and activation functions. NNs are the foundation of modern deep learning and AI.

```
Neuron Output: y^ = f( Σ wᵢxᵢ + b )      where f = activation function, w = weights, b = bias
```

Neural Network Architecture Diagram

```
INPUT LAYER    HIDDEN LAYER 1    HIDDEN LAYER 2    OUTPUT LAYER
(3 neurons)     (4 neurons)       (4 neurons)       (2 neurons)

  [x1] -------->[h11]-------->[h21]                [y1]
      \  \     /    \  \   /    \   \           /
  [x2] --------->[h12]-------->[h22]  --------->
      /  \  \  /    /  \  \  /    /            \
  [x3] -------->[h13]-------->[h23]                [y2]
            \  /         \  /   \
            [h14]        [h24]
(fully connected: each neuron connects to ALL neurons in next layer)
```

Domain	Application
Computer Vision	Image classification, Object detection, Face recognition, Medical scan analysis
NLP	Machine translation, Sentiment analysis, Text generation, Chatbots (GPT)
Speech	Speech recognition (Siri, Alexa), Text-to-speech, Voice cloning (ElevenLabs)
Healthcare	Drug discovery, Protein structure prediction (AlphaFold), Disease diagnosis
Finance	Stock price prediction, Fraud detection, Algorithmic trading
Gaming	Game-playing agents (AlphaGo, Atari DQN), Procedural level generation

Advantages	Limitations
Learns complex non-linear patterns from raw data	Requires very large amounts of labelled training data
Works directly with raw inputs (images, text, audio)	Computationally expensive — needs GPU/TPU hardware
Highly adaptable — same architecture for many task types	Black box — difficult to interpret and explain decisions
State-of-the-art on image, text, and speech tasks	Long training time — can take hours, days, or weeks

Domain	Application
Automatic feature engineering through hidden layers	Prone to overfitting without proper regularisation

Q37. Explain Topologies of Neural Networks. [8 Marks]

Neural Network Topologies

1. **Feedforward Neural Network (FNN / MLP):** Simplest topology. Information flows strictly one way: Input → Hidden → Output. No loops or cycles. Applications: Classification, regression, function approximation.
2. **Convolutional Neural Network (CNN):** Designed for grid-like data (images). Convolutional layers apply learnable filters to detect local patterns (edges, textures, shapes). Uses parameter sharing and pooling. Applications: Image classification (ResNet, VGG), Object detection (YOLO), Medical imaging.
3. **Recurrent Neural Network (RNN):** Designed for sequential data. Feedback connections allow output to be fed back as input for the next time step. Maintains hidden state "memory". Problem: Vanishing gradient for long sequences. Applications: Language modelling, speech recognition.
4. **Long Short-Term Memory (LSTM):** Special RNN solving the vanishing gradient problem via three gates: Forget Gate (what to discard), Input Gate (what to store), Output Gate (what to output). Learns long-range dependencies. Applications: Machine translation, music generation, sentiment analysis.
5. **Autoencoder:** Unsupervised NN with an Encoder that compresses input to latent space and Decoder that reconstructs it. Learns compressed representations. Applications: Dimensionality reduction, anomaly detection, image denoising, data generation.
6. **Generative Adversarial Network (GAN):** Two competing networks: Generator (creates fake data) vs Discriminator (distinguishes real from fake). They train in a minimax adversarial game. Applications: Image synthesis (DALL-E style), deepfakes, drug molecule design, data augmentation.

```
TOPOLOGY COMPARISON:

FNN:  x ->[H1]->[H2]-> y          (one direction)

RNN:  x ->[H] ----------+-> y     (loop: output fed back as input)
             ^           |
             +-----------+

CNN: [Conv]->[ReLU]->[Pool] ×3 ->[FC]-> y  (filter over spatial data)

AUTOENCODER: x ->[Encode]-> z ->[Decode]-> x'  (z = compressed)

GAN: noise ->[Generator]-> fake_x
            real_x  ---->[Discriminator]-> Real/Fake?
```

Q38. Explain Perceptron. [4 Marks]

Perceptron

The Perceptron is the simplest artificial neural network, developed by Frank Rosenblatt in 1958. It is a single artificial neuron for binary classification — classifying inputs into two classes (0 or 1). It is the fundamental building block of all modern neural networks.

Perceptron Architecture

```
          Inputs     Weights    Summation  Activation   Output
                                Function   Function
 x1 (1) ----w1----\
                   \
 x2 (0) ----w2-----+----> [ z = Sw_i*x_i + b ] ---> [ f(z) ] ---> y (0 or 1)
                   /
 x3 (1) ----w3----/
            ^
       bias (b) is added

 Step Activation: f(z) = 1 if z > 0, else 0
```

$$z = w_1 x_1 + w_2 x_2 + \ldots + w_n x_n + b = \Sigma w_i x_i + b$$

$$\text{Output} = 1 \text{ if } z > 0 \quad | \quad \text{Output} = 0 \text{ if } z \leq 0$$

Perceptron Learning Algorithm

1. Initialise weights (w) and bias (b) to small random values or zeros.
2. For each training example (x_i, y_i): compute predicted output $\hat{y} = f(\Sigma w_i x_i + b)$.
3. Calculate error: $e = y_i - \hat{y}$ (actual minus predicted).
4. Update weights: $w_new = w + \eta \times e \times x$ (where η = learning rate, typically 0.01–0.1).
5. Update bias: $b_new = b + \eta \times e$
6. Repeat for all training examples until error ≈ 0 or maximum iterations reached.

Note: *Perceptron Convergence Theorem: If data is linearly separable, the algorithm is guaranteed to converge in finite steps. Limitation: Cannot learn XOR (not linearly separable). Solution: Multi-Layer Perceptron (MLP).*

Q39. Explain Feedforward Neural Network. [8 Marks]

Feedforward Neural Network (FNN / MLP)

A Feedforward Neural Network (also called Multi-Layer Perceptron) is the most fundamental deep learning architecture. Information flows STRICTLY in one direction: input → hidden layer(s) → output. No feedback loops, cycles, or recurrent connections.

Layer	Description
Input Layer	Receives raw input data. Has one neuron per input feature. No computation performed.
Hidden Layers	One or more intermediate layers. Each neuron: $z = \Sigma w_i x_i + b$, then $a = f(z)$. Learns complex feature representations.
Output Layer	Produces final predictions. Neurons = output classes (classification) or 1 (regression).

Activation Functions

Function	Formula	Range	Use Case
Sigmoid	$f(z) = 1/(1+e^{-z})$	(0, 1)	Binary classification output
Tanh	$f(z) = (e^{z}-e^{-z})/(e^{z}+e^{-z})$	(−1, 1)	Hidden layers (zero-centred)
ReLU	$f(z) = \max(0, z)$	$[0, \infty)$	Most common hidden layer activation

Function	Formula	Range	Use Case
Leaky ReLU	$f(z) = \max(0.01z, z)$	$(-\infty, \infty)$	Solves Dying ReLU problem
Softmax	$f(z_i) = e^{z_i} / \Sigma e^{z_j}$	(0,1), sums to 1	Multi-class output (probabilities)

Forward Propagation

```
Layer 1: z⁻¹ = W⁻¹X + b⁻¹  |  a⁻¹ = f(z⁻¹)
Layer 2: z⁻² = W⁻²a⁻¹ + b⁻²  |  y^ = f(z⁻²)
```

Backpropagation (Training)

```
Loss (Cross-Entropy): L = -Σ yᵢ log(y^ᵢ)
Weight Update:        W = W - η × ∂L/∂W   [chain rule of calculus]
```

Backpropagation computes the gradient of the loss with respect to EVERY weight in the network using the chain rule, propagating error from output back to input. Weights are then updated by gradient descent.

Q40. Explain Multi-Layer Perceptron (MLP). [5 Marks]

Multi-Layer Perceptron (MLP)

A Multi-Layer Perceptron has at least three layers: input, one or more hidden, and output. Unlike the single-layer perceptron (solves only linearly separable problems), MLP can solve NON-LINEARLY SEPARABLE problems (like XOR) through hidden layers with non-linear activation functions.

```
Each neuron: aᵢ = f( Σ wᵢⱼ aⱼ + bᵢ )
```

Why Multiple Layers? Hierarchical Feature Learning

- **Layer 1:** Learns simple low-level features (edges, basic patterns in data).
- **Layer 2:** Combines simple features into more complex representations (shapes, corners).
- **Layer 3+:** Learns high-level abstractions (objects, concepts, semantic meaning).

Training Process

1. Initialise all weights randomly (He init for ReLU, Xavier for Sigmoid/Tanh).
2. Forward pass: compute predictions layer by layer through the network.
3. Compute loss using appropriate loss function (Cross-Entropy or MSE).
4. Backward pass: compute gradients using backpropagation and chain rule.
5. Update weights using chosen optimiser (Adam, SGD+Momentum, RMSProp).
6. Repeat for multiple epochs until loss converges or early stopping triggers.

Aspect	Single Perceptron	Multi-Layer Perceptron (MLP)
Layers	One layer (input + output only, no hidden layers)	Input + one or more Hidden + Output layers
Problems Solvable	Only linearly separable: AND, OR (not XOR)	Any complex non-linear problem: XOR, image recognition
Activation Functions	Step function only	Any non-linear: ReLU, Sigmoid, Tanh, Softmax

Aspect	Single Perceptron	Multi-Layer Perceptron (MLP)
Learning Rule	Simple Perceptron Learning Rule	Backpropagation + Gradient Descent optimizer
Power	Limited (single linear decision boundary)	Universal function approximator (can learn any function)

Q41. Explain CNN (Convolutional Neural Network). [8 Marks]

Convolutional Neural Network (CNN)

CNN is a deep learning architecture designed for grid-like data (primarily images). CNNs automatically learn spatial hierarchies of features through convolutional layers with learnable filters, revolutionising computer vision since AlexNet in 2012.

CNN Architecture Diagram

```
Input      Conv+ReLU    Pooling   Conv+ReLU    Pooling  FC    Output
Image                                                    Layer
224x224x3 -> 222x222x32 -> 111x111 -> 109x109x64 -> 54x54 -> 1024 -> 1000
            (32 filters,    (2x2       (64 filters,   (2x2   (Dense) (Softmax)
            3x3 kernels)    MaxPool)   3x3 kernels)   MaxPool)

One filter operating on input:
+-------+     +---+---+---+
| Input |  *  | 1 | 0 | 1 |  =  Dot product at each position
| patch | <-- | 0 | 1 | 0 |     -> one value in feature map
|       |     | 1 | 0 | 1 |
+-------+     +---+---+---+
              3x3 filter/kernel (learnable weights)
```

Layer	Function	Key Parameters
Convolutional Layer	Applies learnable filters to input, creating feature maps detecting edges, textures, patterns	Filter size (3×3, 5×5), stride, padding, #filters
ReLU Activation	f(z)=max(0,z) element-wise. Adds non-linearity after convolution	None
Max Pooling Layer	Reduces spatial dimensions (downsampling). Takes maximum in each window	Pool size (2×2), stride
Batch Normalization	Normalises layer outputs. Speeds training and prevents internal covariate shift	γ, β (learnable scale and shift)
Dropout Layer	Randomly zeros neurons during training. Prevents overfitting.	Dropout rate (e.g., 0.5)
Fully Connected (Dense)	Flattened feature maps fed to standard MLP for final classification	Number of neurons, activation
Softmax Output	Converts logits to class probabilities for multi-class classification	None

```
Output size = (Input − Filter + 2×Padding) / Stride + 1
```

Key Advantages of CNNs

- **Parameter Sharing:** Same filter applied across entire image → far fewer parameters than fully connected networks.
- **Translation Invariance:** Detects features regardless of their position in the image.
- **Hierarchical Learning:** Automatically learns low-level (edges) to high-level features (objects).

Famous CNN Architectures

Architecture	Year	Achievement
LeNet-5	1998	First CNN for digit recognition on MNIST
AlexNet	2012	Won ImageNet with 15.3% top-5 error. Launched deep learning revolution.
VGGNet	2014	Very deep (16–19 layers) with 3×3 filters only
ResNet-50	2015	Residual connections. Achieved 3.57% error — surpassed human (5%)
YOLO	2016+	Real-time object detection in single pass (You Only Look Once)

Q42. Explain RNN (Recurrent Neural Network). [8 Marks]

Recurrent Neural Network (RNN)

RNN is designed for SEQUENTIAL data where the order and context of data points matter. Unlike feedforward networks, RNNs have FEEDBACK CONNECTIONS — the hidden state at time t is fed as input at time t+1, creating a "memory" of past inputs.

```
Hidden state: h(t) = f(Wᵀ × h(t-1) + Wˣ × x(t) + b)

Output:       y(t) = g(Wʸ × h(t) + bʸ)
```

RNN Unfolded Through Time

```
FOLDED RNN:                        UNFOLDED (shown for 4 time steps):

  +------+                  x1    x2    x3    x4
  |      |-->h-->y          |     |     |     |
  +--+---+           [h0]->[h1]->[h2]->[h3]->[h4]
     |  ^                   |     |     |     |
     +--+                  y1    y2    y3    y4
  (loop: h feeds back)  (same weights W reused at every time step)

h(t) = tanh( W_h*h(t-1) + W_x*x(t) + b )
```

RNN Type	Description	Application
One-to-One	Single input → Single output (standard NN)	Image classification
One-to-Many	Single input → Sequence output	Image captioning: image → description words
Many-to-One	Sequence input → Single output	Sentiment analysis: sentence → positive/negative

RNN Type	Description	Application
Many-to-Many (equal)	Sequence → Sequence (same length)	Video frame-by-frame labelling
Many-to-Many (encoder-decoder)	Sequence → Sequence (different lengths)	Machine translation: English → French

Problems with Vanilla RNN — and Solutions

- **Vanishing Gradient:** Gradients shrink exponentially during backpropagation through long sequences → fails to learn long-range dependencies. Solution: LSTM, GRU.
- **Exploding Gradient:** Gradients grow exponentially → unstable training, NaN weights. Solution: Gradient Clipping (cap gradient norm at threshold).

LSTM — Solution to Vanishing Gradient

```
LSTM Cell (Long Short-Term Memory):

   c(t-1) ---[x forget]---[+ input]-----> c(t)  (cell state: long memory)
                  |            |
             [Forget Gate]  [Input Gate]  [Output Gate]
                f(t)            i(t)          o(t)
                 |               |             |
h(t-1) & x(t) --> sigmoid --> tanh --> sigmoid --> h(t)   (hidden state)

Forget Gate: decides what to REMOVE from cell state
Input Gate:  decides what NEW info to ADD to cell state
Output Gate: decides what to OUTPUT as hidden state
```

Q43. Define: Training, Testing and Validation sets. [8 Marks]

Training, Validation, and Test Sets

In ML, the dataset is divided into three distinct subsets. Each serves a different purpose to ensure the model learns well, is tuned correctly, and its performance estimate is unbiased.

Dataset Split Diagram

```
Full Dataset (100%)
+------------------+-----+-----+
|  TRAINING SET    | VAL | TEST|
|   (70%)          |(15%)|(15%)|
+------------------+-----+-----+
      |              |      |
Model trains    Tune HP   Final Eval
weights here    (once)    (once only!)
      |              |      |
Multiple epochs  Model    Unbiased
 of learning    selection  accuracy
```

```
Typical Split: Train = 70-80%  |  Validation = 10-15%  |  Test = 10-15%
```

Aspect	Training Set	Validation Set	Test Set
Purpose	Train model parameters (weights/biases)	Tune hyperparameters and detect overfitting	Final unbiased performance evaluation
Model learns?	YES — weights ARE updated from this data	NO — weights NOT updated (only evaluated)	NO — weights NOT updated

Aspect	Training Set	Validation Set	Test Set
Typical Size	60–80% of total data	10–20% of total data	10–20% of total data
Used How Many Times?	Many times (multiple epochs)	Many times during model development	ONCE at the very end — never during development
Risk of Leakage?	No concern	Risk if used too much to guide decisions	HIGH risk — must be kept strictly separate always
Alternative?	—	K-Fold Cross Validation replaces it	Final holdout set — never replace

 Note: *Critical Rule: NEVER look at test set results during model development. Using test set multiple times causes data leakage and gives an optimistically biased performance estimate.*

Q44. Define Parameter Estimation: MLE, MAP, Bayesian Estimation. [5 Marks]

Parameter Estimation

Parameter Estimation finds optimal values of model parameters (θ) that best explain the observed data. The three main approaches differ in how they treat prior knowledge and uncertainty.

1. Maximum Likelihood Estimation (MLE)

MLE finds θ that maximises the probability (likelihood) of observing the training data. No prior knowledge is used. Only the data counts.

```
θ_MLE = argmax_θ P(X|θ) = argmax_θ Σ log P(xᵢ|θ)
```

Example: For Gaussian model, MLE gives: mean = sample mean, variance = sample variance.

2. Maximum A Posteriori (MAP) Estimation

MAP extends MLE by incorporating PRIOR KNOWLEDGE about parameters P(θ). It finds θ that maximises the posterior probability using Bayes' theorem.

```
θ_MAP = argmax_θ P(θ|X) = argmax_θ [P(X|θ) × P(θ)]
```

Key insight: MAP = MLE when the prior P(θ) is UNIFORM (all values equally likely). Ridge regression (L2 regularisation) is equivalent to MAP with a Gaussian prior.

3. Bayesian Estimation (Full Bayesian Inference)

Unlike MLE and MAP (single point estimates), Bayesian estimation computes the FULL POSTERIOR DISTRIBUTION P(θ|X) over all possible parameter values. Provides complete uncertainty quantification.

```
P(θ|X) = P(X|θ) × P(θ) / P(X)      [Full Bayes formula]

Prediction: P(y|X) = ∫ P(y|X,θ) P(θ|X) dθ      [Integrate over all θ]
```

Aspect	MLE	MAP	Bayesian
Output	Single point estimate	Single point estimate (with prior)	Full probability distribution over θ
Uses Prior?	No	Yes — P(θ)	Yes — P(θ)
Approach	Maximise P(X|θ)	Maximise P(θ|X)	Compute full P(θ|X)
With Uniform Prior	Baseline result	Same as MLE	Same as MLE

Aspect	MLE	MAP	Bayesian
With Little Data	Can overfit (no regularisation)	More robust (prior helps regularise)	Most robust (full uncertainty captured)
Computation	Fast	Fast	Slow (requires numerical integration)
Example	Fitting Gaussian to data	Ridge regression = Gaussian prior	Bayesian neural networks

▶ UNIT 7: DEEP LEARNING

Q45. What is Deep Learning? List applications and explain any one. [8 Marks]

Deep Learning

Deep Learning is a subfield of Machine Learning that uses Artificial Neural Networks with MULTIPLE LAYERS (deep architectures) to automatically learn hierarchical representations from raw data. The "deep" refers to the large number of hidden layers.

```
  Traditional ML: Raw Data -> Manual Feature Engineering -> ML Model ->
Output

  Deep Learning:  Raw Data -> [Layer1->Layer2->...->LayerN] -> Output
(automatic features!)
```

Deep Learning Hierarchy in AI

```
  +-----------------------------------------------+
  |           ARTIFICIAL INTELLIGENCE             |
  |   +-----------------------------------+       |
  |   |        MACHINE LEARNING           |       |
  |   |   +-------------------------+     |       |
  |   |   |     DEEP LEARNING       |     |       |
  |   |   |  +------------------+   |     |       |
  |   |   |  | Neural Networks  |   |     |       |
  |   |   |  | CNN, RNN, LSTM   |   |     |       |
  |   |   |  | Transformers     |   |     |       |
  |   |   |  +------------------+   |     |       |
  |   |   +-------------------------+     |       |
  |   +-----------------------------------+       |
  +-----------------------------------------------+
```

Domain	Application	Model/System
Computer Vision	Image classification, Object detection, Face recognition	ResNet, YOLO, FaceNet
NLP	Text generation, Translation, Chatbots, Summarisation	GPT-4, BERT, T5, LLaMA
Speech	Speech recognition, Text-to-speech, Voice cloning	Whisper, WaveNet, Tacotron
Healthcare	Disease diagnosis, Drug discovery, Protein folding	AlphaFold, PathAI, Med-PaLM
Autonomous Driving	Lane detection, Object tracking, Path planning	Tesla FSD, Waymo
Art & Creativity	Image generation, Music composition, Video synthesis	DALL-E 3, Stable Diffusion, Sora
Recommendation	Content recommendation, Ad targeting	YouTube, Netflix, Spotify algorithms

Detailed Explanation: Image Classification with CNN

CNNs are trained on labelled image datasets to predict which class (category) each image belongs to. The CNN processes images through convolutional layers that detect increasingly complex features: first

layer detects edges and colours; middle layers detect shapes and textures; final layers detect high-level objects. Fully connected layers classify the image.

- **ResNet-50 on ImageNet:** Trained on 1.2M images across 1000 classes. Achieves 94%+ top-5 accuracy. In 2015, ResNet surpassed human-level accuracy (5%) for the first time.
- **Key Innovation — Residual Connections:** y = F(x) + x (skip connection). Allows training 100+ layer networks without vanishing gradients.

Q46. Write down the steps to build an ANN. [8 Marks]

Steps to Build an Artificial Neural Network

1. **Problem Definition:** Define the task: classification, regression, or generation. Determine input features, output format, evaluation metrics, and performance requirements. Example: Build an ANN to classify handwritten digits (0–9) with >98% accuracy.
2. **Data Collection:** Gather sufficient labelled data. Deep learning typically needs thousands to millions of examples. Use datasets like MNIST, CIFAR-10, ImageNet, or collect domain-specific data.
3. **Data Preprocessing:** Clean data (handle missing values, remove duplicates). Normalise/standardise features (scale pixel values 0–255 to 0–1). Split into Train (70%), Validation (15%), Test (15%). Apply data augmentation if needed (rotation, flipping, cropping for images).
4. **Network Architecture Design:** Decide number of layers and neurons per layer. Choose activation functions (ReLU for hidden, Sigmoid/Softmax for output). Set input layer size = number of features; output layer size = number of classes.

```
  Example: Input(784) -> Dense(512, ReLU) -> Dense(256, ReLU) -> Dense(10,
Softmax)
```

5. **Weight Initialisation:** He initialisation for ReLU: $w \sim N(0, \sqrt{(2/n)})$. Xavier initialisation for Sigmoid/Tanh: $w \sim N(0, \sqrt{(1/n)})$. Prevents vanishing/exploding gradients at the start of training.
6. **Define Loss Function:** Classification: Cross-Entropy Loss = $-\Sigma\, y_i \log(\hat{y}_i)$. Regression: Mean Squared Error = $\Sigma(y_i-\hat{y}_i)^2/n$. Binary: Binary Cross-Entropy.
7. **Choose Optimiser:** Adam (most popular, η=0.001). SGD with Momentum for production fine-tuning. RMSProp for RNNs. Set learning rate and other hyperparameters.
8. **Training (Forward + Backward Pass):** Run forward pass → compute predictions. Calculate loss. Backpropagate to compute gradients. Update weights: $w = w - \eta \times \partial L/\partial w$. Repeat for all mini-batches × epochs.
9. **Monitor and Tune:** Track training and validation loss each epoch. Apply regularisation (Dropout, L2) if overfitting detected. Use early stopping: halt when validation loss stops improving. Adjust learning rate (ReduceLROnPlateau).
10. **Evaluate and Deploy:** Evaluate final model on test set ONCE. If satisfactory, export model (SavedModel, ONNX, TorchScript). Monitor in production and retrain periodically as data distribution shifts.

Q47. Define Optimisers. List down the types of optimisers. [8 Marks]

Optimisers

Optimisers are algorithms that update model parameters (weights and biases) during training to minimise the loss function. They determine the direction and magnitude of weight updates based on gradients of the loss.

```
  General Update Rule: θ_new = θ_old − η × ∇L(θ)      where η = learning rate, ∇L = gradient
```

Optimiser	Key Idea	Formula	Pros / Cons
Batch Gradient Descent	Use ALL data to compute gradient before one update	$\theta = \theta - \eta \times (1/n)\Sigma\nabla L(x_i,\theta)$	Pro: Stable convergence. Con: Very slow for large datasets
SGD	Update after EACH single training example	$\theta = \theta - \eta \times \nabla L(x_i,\theta)$	Pro: Fast updates. Con: Very noisy; oscillates around minimum
Mini-Batch GD	Update after each mini-batch of 32–256 samples	$\theta = \theta - \eta \times \nabla L(X_batch,\theta)$	Pro: Best balance of speed and stability. Most common.
Momentum	Accumulate velocity in gradient direction	$v = \beta v - \eta\nabla L$; $\theta = \theta + v$	Pro: Faster convergence, reduces oscillations. Con: Needs β tuning
Nesterov (NAG)	Look-ahead before computing gradient	Uses $\theta+\beta v$ for gradient eval	Pro: More accurate than standard Momentum; slightly better
AdaGrad	Adapts learning rate per parameter; smaller for frequent params	$\theta = \theta - (\eta/\sqrt{(G+\varepsilon)})\nabla L$	Pro: Good for sparse data (NLP). Con: LR shrinks to 0 eventually
RMSProp	Fixes AdaGrad using moving average of squared gradients	$\theta = \theta - (\eta/\sqrt{(E[g^2]+\varepsilon)})g$	Pro: Handles non-stationary objectives. Good for RNNs.
Adam (Most Popular)	Combines Momentum + RMSProp. Adapts LR using 1st and 2nd moments	$m=\beta_1 m+(1-\beta_1)g$; $v=\beta_2 v+(1-\beta_2)g^2$; $\theta=\theta-\eta\hat{m}/(\sqrt{\hat{v}}+\varepsilon)$	Pro: Fast, works well for most problems. Default choice.
AdaMax	Adam variant using infinity norm for 2nd moment	Uses max operation instead of L2 norm	Pro: More stable than Adam in some cases

 Note: *Recommended defaults: Adam with $\eta=0.001$, $\beta_1=0.9$, $\beta_2=0.999$, $\varepsilon=1e-8$. Use SGD+Momentum for fine-tuning large models when production accuracy matters.*

Q48. Explain Gradient Descent. [8 Marks]

Gradient Descent

Gradient Descent is the fundamental optimisation algorithm for training ML/DL models. It minimises the loss function by iteratively moving parameters in the direction of the STEEPEST DESCENT (negative gradient). Analogy: a ball rolling downhill always moves in the direction of steepest slope.

```
  θ_new = θ_old − η × ∂L/∂θ      where η = learning rate, ∂L/∂θ = gradient
```

Gradient Descent Visualisation

```
Loss L(θ)
 |
 |  *   Too high η: overshoots -> diverges!
 | * *  * *
 |   * *   * * *
 |        \
 |     ___  \___
 |   /    \ Just right η: smooth descent to minimum
 |  /      \_____________________________________
 | /       ^minimum (gradient = 0)
 |/
 +----------------------------------------> θ (parameter)
Too low η: tiny steps, very slow, may get stuck
```

Gradient Descent — Step by Step

1. Initialise: Start with random parameter values θ_0 .
2. Compute Loss: Calculate loss function L(θ) using current parameters.
3. Compute Gradient: Calculate ∂L/∂θ — partial derivative of loss with respect to each parameter.
4. Update Parameters: Move in negative gradient direction: θ = θ − η × ∂L/∂θ.
5. Repeat steps 2–4 until convergence (loss stops decreasing significantly or max iterations reached).

Effect of Learning Rate (η)

Learning Rate	Effect
Too High (η >> 1)	Takes large steps → overshoots minimum → loss oscillates or diverges (training fails completely)
Too Low (η << 0.001)	Tiny steps → extremely slow convergence → may get stuck in local minima or saddle points
Just Right (η ≈ 0.001)	Steady, consistent convergence → reaches minimum efficiently in reasonable time

Types of Gradient Descent

Type	Batch Size	Updates/Epoch	Speed	Stability
Batch GD	All n samples	1 update	Slow	Very stable, smooth loss curve
Stochastic GD (SGD)	1 sample	n updates	Very fast	Noisy; can escape local minima
Mini-Batch GD	32–256 samples	n/batch updates	Fast (best in practice)	Good balance of speed and stability

Challenges

- **Local Minima:** May converge to a local minimum instead of global minimum (less of an issue in deep learning due to many saddle points).
- **Saddle Points:** Gradient = 0 but not at a minimum; algorithm stalls. Solved by Momentum and Adam.

- **Vanishing Gradients:** In deep networks, gradients become very small in early layers → no learning. Solved by ReLU and Batch Normalization.
- **Exploding Gradients:** Gradients grow very large → unstable training. Solved by gradient clipping and careful initialisation.

Q49. Explain Stochastic Gradient Descent (SGD). [4 Marks]

Stochastic Gradient Descent (SGD)

SGD is a variant of gradient descent that updates model parameters after processing EACH INDIVIDUAL TRAINING EXAMPLE, rather than after the entire dataset. "Stochastic" means random — a random single sample is chosen at each step.

```
SGD Update Rule: θ = θ − η × ∇L(xᵢ, yᵢ; θ)   where (xᵢ, yᵢ) is ONE random training example
```

SGD vs Batch GD Path Visualisation

```
Loss Contour (top view):      x = position, * = minimum

Batch GD:        SGD:              Mini-Batch GD:
+----------+     +----------+      +----------+
|    *     |     |    *     |      |    *     |
|   /      |     | /\/\/\ * |      | /~\~\/   |
|  /       |     |/    \  \ |      |/    \    |
| /        |     |      \  ||      |      \   |
|/         |     |       \/ |      |       \  |
+----------+     +----------+      +----------+
Smooth path       Very noisy        Moderate noise
but 1 step        fast updates      best balance
per epoch         many per epoch
```

Feature	Batch GD	SGD	Mini-Batch GD
Updates per Epoch	1 (after ALL data)	n (after EACH sample)	n/batch_size (after each batch)
Gradient Accuracy	Exact gradient computation	Very noisy approximation	Good approximation of true gradient
Memory Usage	High (full dataset in RAM)	Very low (one sample)	Moderate (one batch in RAM)
Convergence	Smooth but slow	Noisy but fast; escapes local minima	Fast and relatively smooth
Practical Use	Rarely used for large data	Conceptual; rarely used in practice	Most common in deep learning

SGD with Momentum

Momentum adds a fraction of the previous update direction to the current one. This accelerates convergence in relevant directions and dampens oscillations in irrelevant directions.

```
v = β × v_prev + η × ∇L(θ)      [β = momentum coefficient, typically 0.9]
θ = θ − v
```

When to Use SGD

- Works well when data is too large to fit entirely in memory (online learning setting).

- The noisy updates help escape local minima — beneficial for complex deep networks.
- SGD + Momentum + LR scheduling can match or beat Adam for final model accuracy.
- Used in large-scale production training: ImageNet training uses SGD + Momentum.